Map out a complete semester administration console: term scheduling, academic calendars, and role-based permissions in one dashboard

Computer
Build an institution operations console blueprint: department hierarchies, registrar controls, and FERPA audit workflows in a unified spec
Computer
Show the screen-by-screen admin console IA
Build a module-by-module requirements doc
Map out the database and domain architecture
Artifacts
18
Sources
325

Semester Institution OS should be designed as a unified **administration console** that combines academic operations, registrar workflows, student-life operations, permissions, and compliance into one governed control plane. Higher-ed scheduling and registrar tools emphasize department-level schedule submission, room/resource planning, and SIS-connected workflows, while FERPA guidance requires role-based access, legitimate educational interest, and auditable record access. [1] [2] [3]

Screen-by-screen IA

The admin console should have one global shell with scoped navigation, a campus/term selector, alerts, search, and an audit-aware activity tray. Role-based access should determine not only which pages are visible but also which fields, actions, exports, and override tools appear. [4] [5]

Global navigation

```
Home
Operations
Academic Operations
Registrar
Student Life
People & Orgs
Governance
Integrations
```

```
Analytics
Admin
```

1. Home dashboard

Purpose: executive and operator overview.

Widgets

- Current term status, registration windows, calendar milestones.
- Queue summary: approvals, conflicts, incidents, export reviews.
- Academic health: unassigned sections, room conflicts, schedule changes.
- Engagement health: club activity, event registrations, marketplace application counts.
- Compliance health: unusual access, pending audit reviews, recent exports.
- Integration health: SIS sync, LMS sync, email/push delivery, SSO status.

2. Operations Center

Purpose: action-first queue management.

Subscreens

- Work queues.
- Assigned to me.
- Escalations.
- Exceptions and overrides.
- Bulk operations.
- SLA and aging.

Common queue types

- Calendar changes awaiting approval.
- Section edits awaiting registrar review.
- Department requests.
- Club approvals.
- Marketplace moderation.
- Study-abroad document exceptions.
- Privacy/access incidents.

3. Academic Operations

Purpose: term scheduling, academic calendar, rooming, and section lifecycle.

Subscreens

- Term planner.
- Academic calendar builder.
- Schedule of classes.
- Section conflict monitor.
- Room and resource planner.
- Faculty load and assignment.
- Change history.

4. Registrar

Purpose: authoritative controls over academic structure and official records workflows.

Subscreens

- Catalog and course inventory.
- Section governance.
- Registration windows and rules.
- Add/drop and withdrawal policy controls.
- Grade deadlines and submission monitoring.
- Official communications and notices.
- Exception approvals.

5. Student Life

Purpose: non-academic engagement operations.

Subscreens

- Clubs and organizations.
- Events and RSVP oversight.
- Marketplace listings.
- Study abroad.
- Social/community moderation.
- Messaging campaigns.

6. People & Organizations

Purpose: person, office, and organization master records.

Subscreens

- Students.
- Staff and advisors.
- Department hierarchy.
- Offices and service units.
- Clubs and official organizations.
- External partners/providers.

7. Governance

Purpose: permissions, audits, FERPA, retention, and policy.

Subscreens

- Roles and permission templates.
- Scoped access assignments.
- Audit log explorer.
- Export review.
- Consent and disclosure records.
- Data retention and legal hold.
- Incident response.

8. Integrations

Purpose: connected systems.

Subscreens

- SIS.
- LMS.
- SSO/identity.
- Calendar and rooming feeds.
- Email/SMS/push.
- Career/provider systems.
- Webhooks and API clients.

9. Analytics

Purpose: governed reporting.

Subscreens

- Executive dashboard.
- Registrar analytics.
- Scheduling analytics.
- Student-life analytics.
- Funnel analytics.
- Equity/access reporting, if approved.
- Audit/compliance reporting.

10. Admin

Purpose: tenant-wide configuration.

Subscreens

- Branding and campus structure.
- Taxonomy.
- Feature flags.
- Environment settings.
- Sandboxes.
- Notification templates.
- Support tools.

Module-by-module requirements doc

A. Term scheduling and academic calendar

Academic scheduling systems in higher education typically support demand-aware planning, department-level schedule submission, room/resource alignment, and academic/event calendar publishing. [6] [1]

Requirements

- Create academic years, terms, parts of term, sessions, and milestone windows.
- Support semester, quarter, compressed, and year-round models.
- Build master academic calendar with registration, holidays, exam periods, grading deadlines, and institutional milestones.
- Allow department-proposed schedules with registrar review.

- Track rooms, capacities, equipment, accessibility constraints, and special-use spaces.
- Detect conflicts in instructor time, room use, and cross-listed offerings.
- Publish approved schedules to student-facing systems.
- Maintain a full version history of calendar and schedule changes.

Key workflows

- Draft term setup.
- Department schedule proposal.
- Registrar review and conflict resolution.
- Publish schedule.
- Mid-cycle change request.
- Archive completed term.

B. Department hierarchy and organizational control

Parent-child organizational structures are a practical pattern for inherited settings with local overrides, and education admin systems often use hierarchy to assign services and permissions. [7]

Requirements

- Represent institution > campus > college > school > department > program > office.
- Inherit policy defaults from parent nodes.
- Allow scoped overrides at lower levels.
- Attach staff, roles, workflows, and data access to hierarchy nodes.
- Support cross-functional groups without breaking the formal reporting tree.
- Track effective permissions and inherited settings visibly.

C. Registrar controls

Registrar systems commonly centralize designated schedule contacts, class maintenance, and authoritative schedule-of-classes control. [3]

Requirements

- Course and section lifecycle management.
- Department scheduler designation.
- Approval gates for official schedule changes.
- Registration rule windows by student population or cohort.
- Holds, overrides, and exception approvals.
- Grade submission deadlines and compliance reminders.
- Audit trail for all registrar actions.

- Immutable publication snapshots for official term releases.

D. Role-based permissions

RBAC in education should distinguish view, edit, export, and administrative actions, and pair those with logging.[4]

Requirements

- Role templates by function, not generic admin status.
- Scope permissions by campus, department, program, student relationship, and record type.
- Field-level restrictions for sensitive data.
- Time-bound elevated access.
- Delegated administration with approval.
- Preview effective access before applying role changes.
- Require justification for exports, overrides, and emergency access.

Permission dimensions

- View.
- Edit.
- Approve.
- Publish.
- Export.
- Override.
- Configure.
- Assign roles.

E. FERPA audit workflows

FERPA-oriented systems should log who accessed which student records, when, and what action occurred, while enforcing legitimate educational interest and least privilege.[5] [2] [8]

Requirements

- Log authentication, record access, updates, exports, role changes, and overrides.
- Make logs append-only and tamper-evident.
- Filter audits by student, user, office, module, and date.
- Create access reports for a specific student record.
- Flag anomalous access, such as off-scope or unusual-hour access.
- Route sensitive incidents to compliance reviewers.
- Track disclosure and export justifications.

- Support evidence packs for audits and investigations.

F. Student-life operational modules

Clubs and organizations

- Approvals, renewals, constitutions, advisor assignments, roster oversight, event risk review.

Marketplace

- Listing creation, provider verification, category rules, moderation queues, performance analytics.

Study abroad

- Program terms, eligibility, application templates, document review, decisions, pre-departure tracking.

Social/community

- Feed rules, reporting, moderator queues, broadcast controls, escalation cases.

Database and domain architecture

The cleanest architecture is a modular monolith or service-oriented platform with shared identity, policy, workflow, audit, and search layers.

Shared core domains

Domain	Main entities
Identity	`users`, `identities`, `sessions`, `mfa_factors`
Org model	`institutions`, `campuses`, `org_units`, `org_unit_paths`, `groups`
Access	`roles`, `permissions`, `role_bindings`, `access_scopes`, `elevation_grants`
Audit	`audit_events`, `audit_exports`, `access_reviews`, `incident_cases`
Workflow	`work_items`, `approvals`, `transitions`, `sla_policies`, `comments`
Files	`files`, `attachments`, `document_packets`
Notifications	`notification_templates`, `notifications`, `delivery_attempts`

Academic operations domain

Entity	Purpose
`academic_years`	Top-level academic cycle
`terms`	Semester/quarter/session containers
`parts_of_term`	Subterm structures

Entity	Purpose
`academic_calendar_events`	Milestones, deadlines, holidays, exams
`courses`	Catalog records
`course_versions`	Versioned catalog definitions
`sections`	Scheduled class instances
`section_meetings`	Days/times/location patterns
`rooms`	Physical spaces
`room_features`	Capacity, equipment, accessibility
`faculty_assignments`	Instructor-to-section links
`schedule_change_requests`	Controlled edits and approvals

Registrar domain

Entity	Purpose
`registration_windows`	Open/close periods by population
`registration_rules`	Restrictions, priorities, prerequisites
`holds`	Financial/advising/compliance constraints
`override_requests`	Exception workflows
`grade_deadlines`	Submission milestones
`official_publications`	Versioned published schedule snapshots

Student-life domain

Entity	Purpose
`organizations`	Clubs and official groups
`organization_roles`	Officers, advisors
`memberships`	Roster state
`events`	Event records
`rsvps`	Attendance intent
`listings`	Marketplace opportunities
`listing_reviews`	Moderation/approval state
`programs`	Study-abroad catalog
`program_applications`	Student applications
`application_documents`	Files and requirements

Entity	Purpose
`posts`	Social feed content
`conversations`	Messaging threads
`reports`	Moderation or safety reports

Hierarchy model

Use a path-based org hierarchy:

```
Institution
 ├─ Campus
 │   ├─ College
 │   │   ├─ Department
 │   │   │   ├─ Program
 │   │   │   └─ Office
```

Recommended tables:

- `org_units`
- `org_unit_closure` or `org_unit_paths`
- `org_unit_policies`
- `org_unit_memberships`

This supports inherited configuration, scoped access, and roll-up analytics.

Permission model

A useful permission binding shape is:

```
role_binding
  user_id
  role_id
  scope_type   (institution / campus / org_unit / program / record-set)
  scope_id
  start_at
  end_at
  granted_by
  justification
```

Then resolve effective access through:

- Role template.
- Scope inheritance.
- Record relationship.
- Field sensitivity classification.

- Temporary elevations.
- Deny rules and segregation-of-duty policies.

Audit model

Minimum audit event shape:

```
audit_event
  id
  actor_user_id
  acted_as_role_id
  action_type
  module
  target_type
  target_id
  scope_type
  scope_id
  reason_code
  justification
  metadata
  ip_address
  user_agent
  occurred_at
  hash_chain
```

Sources discussing FERPA-compliance tooling consistently highlight audit logs for access, exports, and modifications as core evidence for investigations and reviews. [9] [8] [5]

Screen flows

Term planner

```
Academic Operations
  → Term Planner
  → Select Academic Year
  → Create / edit terms
  → Configure registration, census, exams, grading deadlines
  → Validate conflicts
  → Submit for registrar approval
  → Publish
```

Department schedule submission

```
Department Scheduler
  → Open draft schedule
  → Add / edit sections
  → Assign meeting patterns and rooms
  → Resolve conflicts
  → Submit changes
```

```
  → Registrar review
  → Approved / returned
```

FERPA audit review

```
Governance
  → Audit Log Explorer
  → Filter by student / staff / office / date
  → Open event timeline
  → View access rationale and role scope
  → Mark reviewed / escalate incident / export evidence pack
```

Recommended console principles

- One dashboard shell, many scoped modules.
- Hierarchy-aware data access everywhere.
- Workflow states over ad hoc edits.
- Published snapshots for official records.
- Append-only audit trails for sensitive actions.
- Override powers that are rare, explicit, and justified.
- Clear distinction between operational work queues and authoritative record systems.

In effect, Semester Institution OS becomes a hybrid of registrar console, student-life operations platform, permissions engine, and compliance workspace. Higher-ed scheduling tools show the need for coordinated term, room, and class planning, while FERPA guidance and RBAC best practices require strict scope control and auditable access. [1] [6] [4]

⁂

1. https://www.aais.com/
2. https://studentprivacy.ed.gov/ferpa
3. https://registrar.utah.edu/scheduling/classes/scheduler-onboarding.php
4. https://bluepes.com/blog/role-based-access-education-systems
5. https://openeducat.org/features/ferpa-compliance/
6. https://moderncampus.com/blog/how-to-choose-a-course-scheduling-system-for-your-university.html
7. https://www.canopy.education/post/ten-ways-to-configure-the-google-for-education-admin-console-to-secure-your-domain
8. https://secureprivacy.ai/blog/ferpa-compliance-software
9. https://www.preveil.com/blog/email-file-sharing-ferpa-compliance/
10. https://www.scholaro.com/db/News/universities-changing-academic-calendar-384